



Solemne 2

INDICACIONES

- Esta Solemne se realiza de forma individual.
- La solemne debe ser rendida en la plataforma <http://codetrack.cl/>
- Para entrar debe escribir su correo institucional y como password su rut (solo numeros).
- Lea cada pregunta con atención y programe en la plataforma.
- No se permite el uso de dispositivos electrónicos como celulares, relojes inteligentes, tablets, etc. Quien sea sorprendido/a con alguno de ellos, se le evaluará con la nota mínima.
- Cualquier consulta sobre la plataforma debe ser resuelta con los profesores a cargo.
- Puede usar su cheat sheet impreso y entregado en clases.

Problemas

1. Curva de brillo de una estrella variable.

Enunciado

En astronomía, una estrella variable es una estrella cuyo brillo observado cambia con el tiempo. Una forma simple de modelar estas variaciones es construir una curva de luz, donde se grafica el brillo de la estrella en función del tiempo. En este ejercicio se trabajará con un modelo periódico compuesto por varias oscilaciones sinusoidales, similar a una señal astronómica idealizada.

Considere que t representa el tiempo medido en horas. El brillo relativo de la estrella se modelará mediante la función

$$B(t) = 1 + a_1 \sin\left(\frac{2\pi t}{p}\right) + a_2 \sin\left(\frac{4\pi t}{p} + \phi_1\right) + a_3 \sin\left(\frac{6\pi t}{p} + \phi_2\right),$$

donde los parámetros a_1 , a_2 , a_3 , p , ϕ_1 y ϕ_2 son constantes.

Use los valores

$$a_1 = 0,35, \quad a_2 = 0,18, \quad a_3 = 0,10, \quad p = 4,8, \quad \phi_1 = \frac{\pi}{3}, \quad \phi_2 = -\frac{\pi}{5}.$$

- Cree un arreglo t con 300 valores entre 0 y 12 horas usando `np.linspace`.
- Cree una función llamada `brillo_estrella(t)` que reciba el arreglo de tiempos y retorne el brillo $B(t)$.
- Evalúe la función anterior para obtener un arreglo B con los valores del brillo de la estrella.
- Grafique el brillo B en función del tiempo t usando `matplotlib`. La gráfica debe tener título, etiquetas en los ejes, grilla y una apariencia clara.
- Calcule e imprima el brillo máximo $B_{\text{máx}}$, el brillo mínimo $B_{\text{mín}}$, el brillo promedio $\overline{B}$ y la amplitud de variación

$$A = \frac{B_{\text{máx}} - B_{\text{mín}}}{2}.$$

- Cuente cuántos puntos del arreglo B quedan sobre el brillo promedio $\overline{B}$.
- Use un condicional para clasificar la variabilidad de la estrella como baja o alta. Para ello, defina la variabilidad relativa como

$$V_{\text{rel}} = \frac{A}{\overline{B}}.$$

Si $V_{\text{rel}} \geq 0,25$, la variabilidad se considerará alta. En caso contrario, se considerará baja.

Solución. Para resolver el problema se debe construir primero el arreglo temporal, luego definir una función que represente el modelo de brillo y finalmente analizar numéricamente los valores obtenidos.

El tiempo t se representa mediante un arreglo de NumPy porque se desea evaluar la función $B(t)$ en muchos puntos de forma simultánea. La instrucción `np.linspace(0, 12, 300)` genera 300 puntos igualmente espaciados desde 0 hasta 12, incluyendo ambos extremos.

Los parámetros elegidos para el modelo son:

$$a_1 = 0,35, \quad a_2 = 0,18, \quad a_3 = 0,10, \quad p = 4,8, \quad \phi_1 = \frac{\pi}{3}, \quad \phi_2 = -\frac{\pi}{5}.$$

Estos valores producen una curva de luz suave, periódica y con estructura visual interesante, sin que el brillo relativo tome valores negativos.

La función `brillo_estrella(t)` recibe el arreglo de tiempos y retorna el arreglo de brillos. Como NumPy permite operar directamente con arreglos, no es necesario usar un ciclo para calcular cada valor de $B(t)$: las operaciones sinusoidales se aplican automáticamente a todos los elementos del arreglo.

Luego se calculan las cantidades pedidas:

$$B_{\text{máx}} = \text{máx}(B), \quad B_{\text{mín}} = \text{mín}(B), \quad \overline{B} = \frac{1}{N} \sum_{i=1}^N B_i, \quad A = \frac{B_{\text{máx}} - B_{\text{mín}}}{2}.$$

Para contar los puntos sobre el promedio se usa una comparación elemento a elemento:

$$B_i > \overline{B}.$$

Esta comparación genera un arreglo de valores booleanos. Al aplicar `np.sum`, Python cuenta cuántos valores son verdaderos.

La clasificación final se realiza mediante un condicional `if`. La cantidad

$$V_{\text{rel}} = \frac{A}{\overline{B}}$$

mide qué tan grande es la amplitud de variación comparada con el brillo promedio. Si esta razón es mayor o igual que 0,25, se clasifica como una estrella de variabilidad alta; de lo contrario, como una estrella de variabilidad baja.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parámetros del modelo de brillo
5 a1 = 0.35
6 a2 = 0.18
7 a3 = 0.10
8 p = 4.8
9 phi1 = np.pi / 3
10 phi2 = -np.pi / 5
11
12 # (a) Arreglo de tiempo entre 0 y 12 horas
13 t = np.linspace(0, 12, 300)
14
15 # (b) Función para calcular el brillo de la estrella
16 def brillo_estrella(t):
17     B = (
18         1
19         + a1 * np.sin(2 * np.pi * t / p)
20         + a2 * np.sin(4 * np.pi * t / p + phi1)
21         + a3 * np.sin(6 * np.pi * t / p + phi2)
22     )
23     return B
24
25 # (c) Evaluar el brillo en cada instante de tiempo
26 B = brillo_estrella(t)
27
28 # (d) Graficar brillo versus tiempo
29 plt.figure(figsize=(9, 5))
```

```

30 plt.plot(t, B, linewidth=2, label="Brillo relativo")
31 plt.axhline(np.mean(B), linestyle="--", linewidth=1.5, label="Brillo promedio")
32
33 plt.title("Curva de luz simulada de una estrella variable")
34 plt.xlabel("Tiempo t [horas]")
35 plt.ylabel("Brillo relativo B(t)")
36 plt.grid(True, alpha=0.3)
37 plt.legend()
38 plt.show()
39
40 # (e) Cálculo de cantidades características
41 B_max = np.max(B)
42 B_min = np.min(B)
43 B_promedio = np.mean(B)
44 amplitud = (B_max - B_min) / 2
45
46 print(f"Brillo máximo: {B_max:.3f}")
47 print(f"Brillo mínimo: {B_min:.3f}")
48 print(f"Brillo promedio: {B_promedio:.3f}")
49 print(f"Amplitud de variación: {amplitud:.3f}")
50
51 # (f) Contar puntos sobre el brillo promedio
52 puntos_sobre_promedio = np.sum(B > B_promedio)
53
54 print(f"Puntos sobre el brillo promedio: {puntos_sobre_promedio}")
55
56 # (g) Clasificación de la variabilidad
57 variabilidad_relativa = amplitud / B_promedio
58
59 if variabilidad_relativa >= 0.25:
60     clasificacion = "alta"
61 else:
62     clasificacion = "baja"
63
64 print(f"Variabilidad relativa: {variabilidad_relativa:.3f}")
65 print(f"La estrella tiene variabilidad {clasificacion}.")

```

El programa construye una curva de luz limpia y permite analizar sus propiedades principales. Además, el uso de una función hace que el modelo de brillo quede separado del análisis posterior, lo cual mejora el orden del código y permite modificar fácilmente los parámetros físicos del modelo.

2. Movimiento de un proyectil lanzado desde un edificio.

Enunciado

Un proyectil se lanza desde la parte superior de un edificio de altura inicial y_0 , con rapidez inicial v_0 y un ángulo de lanzamiento θ respecto de la horizontal. Se desea estudiar su movimiento usando arreglos de NumPy y visualizar, en una misma figura, la posición, velocidad y aceleración en las direcciones horizontal y vertical.

Considere los siguientes valores constantes:

$$y_0 = 50 \text{ m}, \quad v_0 = 35 \text{ m/s}, \quad \theta = 55^\circ, \quad g = 9,8 \text{ m/s}^2.$$

El tiempo debe representarse mediante un arreglo temporal construido con `np.linspace`. En este problema use un arreglo `t` con 300 valores entre $t = 0$ y $t = 8$ segundos.

Las ecuaciones cinemáticas del movimiento son:

$$x(t) = v_0 \cos(\theta)t,$$

$$y(t) = y_0 + v_0 \sin(\theta)t - \frac{1}{2}gt^2.$$

Las velocidades en cada dirección están dadas por:

$$v_x(t) = v_0 \cos(\theta),$$

$$v_y(t) = v_0 \sin(\theta) - gt.$$

Las aceleraciones correspondientes son:

$$a_x(t) = 0,$$

$$a_y(t) = -g.$$

Escriba un programa en Python que realice lo siguiente:

- Defina las constantes físicas del problema y construya el arreglo temporal t usando `np.linspace` con inicio 0, final 8 y 300 puntos.
- Calcule los arreglos de posición $x(t)$ y $y(t)$ usando las ecuaciones cinemáticas entregadas.
- Calcule los arreglos de velocidad $v_x(t)$ y $v_y(t)$. Recuerde que, aunque $v_x(t)$ sea constante, debe representarse como un arreglo con la misma dimensión que t para poder graficarlo correctamente.
- Encuentre el índice y el valor máximo de cada arreglo de velocidad, es decir, de $v_x(t)$ y $v_y(t)$. Además, calcule el promedio de ambos arreglos de velocidad.
- Calcule los arreglos de aceleración $a_x(t)$ y $a_y(t)$. Para $a_x(t)$ puede usar un arreglo de ceros con la misma dimensión que el tiempo, por ejemplo mediante `np.zeros_like(t)`. Para $a_y(t)$ debe construir un arreglo constante igual a $-g$.
- Realice una figura con subgráficos usando `plt.subplots(3, 2)`. En la primera columna debe graficar las cantidades asociadas al movimiento vertical:

$$y(t) \text{ vs. } t, \quad v_y(t) \text{ vs. } t, \quad a_y(t) \text{ vs. } t.$$

En la segunda columna debe graficar las cantidades asociadas al movimiento horizontal:

$$x(t) \text{ vs. } t, \quad v_x(t) \text{ vs. } t, \quad a_x(t) \text{ vs. } t.$$

- Agregue títulos, nombres de ejes y una presentación limpia a cada gráfico, de modo que la figura permita comparar claramente la evolución horizontal y vertical del proyectil.

Solución. Una posible solución consiste en construir primero el arreglo temporal, luego calcular las cantidades cinemáticas como arreglos de NumPy y finalmente graficarlas en una figura de 3×2 subgráficos.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Constantes físicas
5 y0 = 50          # altura inicial en metros
6 v0 = 35          # rapidez inicial en m/s
7 theta_deg = 55   # ángulo en grados
8 g = 9.8          # aceleración de gravedad en m/s^2
9
10 # Conversión del ángulo a radianes
11 theta = np.deg2rad(theta_deg)
12
13 # Arreglo temporal
14 t = np.linspace(0, 8, 300)
15
16 # Posiciones
17 x = v0 * np.cos(theta) * t
18 y = y0 + v0 * np.sin(theta) * t - 0.5 * g * t**2
19
20 # Velocidades
21 vx = v0 * np.cos(theta) * np.ones_like(t)
22 vy = v0 * np.sin(theta) - g * t
```

```

23
24 # Índices de velocidad máxima
25 idx_vx_max = np.argmax(vx)
26 idx_vy_max = np.argmax(vy)
27
28 # Valores máximos
29 vx_max = vx[idx_vx_max]
30 vy_max = vy[idx_vy_max]
31
32 # Promedios
33 vx_prom = np.mean(vx)
34 vy_prom = np.mean(vy)
35
36 print("Velocidad máxima en x:")
37 print("Índice:", idx_vx_max)
38 print("Valor:", vx_max)
39
40 print("\nVelocidad máxima en y:")
41 print("Índice:", idx_vy_max)
42 print("Valor:", vy_max)
43
44 print("\nPromedios:")
45 print("Promedio de vx:", vx_prom)
46 print("Promedio de vy:", vy_prom)
47
48 # Aceleraciones
49 ax = np.zeros_like(t)
50 ay = -g * np.ones_like(t)
51
52 # Figura con subgráficos
53 fig, axs = plt.subplots(3, 2, figsize=(10, 9))
54
55 # Columna izquierda: movimiento vertical
56 axs[0, 0].plot(t, y)
57 axs[0, 0].set_title("Posición vertical")
58 axs[0, 0].set_xlabel("t [s]")
59 axs[0, 0].set_ylabel("y(t) [m]")
60 axs[0, 0].grid(True)
61
62 axs[1, 0].plot(t, vy)
63 axs[1, 0].set_title("Velocidad vertical")
64 axs[1, 0].set_xlabel("t [s]")
65 axs[1, 0].set_ylabel("vy(t) [m/s]")
66 axs[1, 0].grid(True)
67
68 axs[2, 0].plot(t, ay)
69 axs[2, 0].set_title("Aceleración vertical")
70 axs[2, 0].set_xlabel("t [s]")
71 axs[2, 0].set_ylabel("ay(t) [m/s²]")
72 axs[2, 0].grid(True)
73
74 # Columna derecha: movimiento horizontal
75 axs[0, 1].plot(t, x)
76 axs[0, 1].set_title("Posición horizontal")
77 axs[0, 1].set_xlabel("t [s]")
78 axs[0, 1].set_ylabel("x(t) [m]")
79 axs[0, 1].grid(True)
80
81 axs[1, 1].plot(t, vx)
82 axs[1, 1].set_title("Velocidad horizontal")
83 axs[1, 1].set_xlabel("t [s]")
84 axs[1, 1].set_ylabel("vx(t) [m/s]")
85 axs[1, 1].grid(True)
86

```

```

87 axes[2, 1].plot(t, ax)
88 axes[2, 1].set_title("Aceleración horizontal")
89 axes[2, 1].set_xlabel("t [s]")
90 axes[2, 1].set_ylabel("ax(t) [m/s²]")
91 axes[2, 1].grid(True)
92
93 plt.tight_layout()
94 plt.show()

```

3. Clasificación de estrellas según su brillo aparente.

Enunciado

En astronomía, el brillo aparente de una estrella puede estimarse a partir de su luminosidad y su distancia al observador. Una forma simplificada de calcular este brillo es mediante la expresión:

$$F = \frac{L}{4\pi d^2},$$

donde F representa el brillo observado, L la luminosidad de la estrella y d la distancia a la estrella. Considere los siguientes arreglos con información de distintas estrellas:

$$L = [120, 80, 15, 5, 200, 40],$$

$$d = [3, 5, 8, 10, 6, 12].$$

Cada elemento de ambos arreglos corresponde a una misma estrella. Es decir, la luminosidad $L[i]$ y la distancia $d[i]$ pertenecen a la estrella ubicada en el índice i .

Escriba un programa en Python que realice lo siguiente:

- Defina los arreglos de luminosidad L y distancia d usando `np.array`.
- Cree una lista vacía donde se almacenarán los valores de brillo F calculados para cada estrella.
- Recorra todos los índices de los arreglos usando un ciclo `for`. En cada iteración, acceda a $L[i]$ y $d[i]$ para calcular el brillo de la estrella correspondiente y agréguelo a la lista, por ejemplo con `lista_vacia.append(calculo_F)`:

$$F_i = \frac{L_i}{4\pi d_i^2}.$$

- Para cada estrella, clasifique su brillo usando condicionales según el siguiente criterio:

$$F > 1 \implies \text{Muy brillante},$$

$$0,1 < F \leq 1 \implies \text{Brillo medio},$$

$$F \leq 0,1 \implies \text{Débil}.$$

- Muestre en pantalla, para cada estrella, su índice, su luminosidad, su distancia, su brillo calculado y su clasificación.
- Calcule e imprima el brillo promedio de todas las estrellas.
- Realice un gráfico de dispersión usando `plt.scatter`, donde el eje horizontal represente la distancia d y el eje vertical represente el brillo aparente F . Agregue título, nombres de ejes, grilla y una presentación limpia de la figura.

El objetivo del ejercicio es practicar el recorrido de arreglos mediante índices, usando expresiones como $L[i]$ y $d[i]$, junto con condicionales para clasificar datos físicos y visualizar los resultados mediante un gráfico de dispersión.

Solución. Una posible solución consiste en recorrer los arreglos usando sus índices. En cada iteración se calcula el brillo de la estrella correspondiente, se guarda el resultado en una lista y se clasifica usando condicionales.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Arreglos de datos
5 L = np.array([120, 80, 15, 5, 200, 40])
6 d = np.array([3, 5, 8, 10, 6, 12])
7
8 # Lista vacía para guardar los brillos
9 brillos = []
10
11 # Recorrido usando índices
12 for i in range(len(L)):
13
14     # Cálculo del brillo aparente
15     F = L[i] / (4 * np.pi * d[i]**2)
16
17     # Guardar el brillo calculado
18     brillos.append(F)
19
20     # Clasificación usando condicionales
21     if F > 1:
22         clasificacion = "Muy brillante"
23     elif F > 0.1:
24         clasificacion = "Brillo medio"
25     else:
26         clasificacion = "Débil"
27
28     # Mostrar resultados de cada estrella
29     print("=" * 40)
30     print("Estrella índice:", i)
31     print("Luminosidad:", L[i])
32     print("Distancia:", d[i])
33     print("Brillo:", F)
34     print("Clasificación:", clasificacion)
35
36 # Convertir la lista de brillos a arreglo de NumPy
37 brillos = np.array(brillos)
38
39 # Brillo promedio
40 brillo_promedio = np.mean(brillos)
41
42 print("=" * 40)
43 print("Brillo promedio:", brillo_promedio)
44
45 # Gráfico de dispersión
46 plt.figure(figsize=(7, 5))
47 plt.scatter(d, brillos)
48
49 plt.title("Brillo aparente de estrellas según su distancia")
50 plt.xlabel("Distancia d")
51 plt.ylabel("Brillo aparente F")
52 plt.grid(True)
53
54 plt.show()

```